



FACE RECOGNITION MLOps SYSTEM

Complete User Manual
From Git Clone to Live Camera Feed

Docker

Airflow

MLflow

PyTorch

Flask

DA5402 — MLOps Final Project
github.com/DextroLaev/DA5402-MLOPS---Final-Project

Table of Contents

1. Prerequisites & System Requirements

- 1.1 Hardware requirements
- 1.2 Software to install
- 1.3 Verifying your installation

2. Cloning the Repository

3. Files You Must Create Yourself

- 3.1 The .env file
- 3.2 SSH key setup (remote training only)
- 3.3 simple_auth_manager_passwords.json

4. Downloading the Dataset

5. Building & Starting All Services

- 5.1 Build Docker images
- 5.2 Start the stack
- 5.3 Verify all services are healthy

6. Running the Training Pipeline (Airflow)

- 6.1 Open the Airflow UI
- 6.2 Trigger the DAG
- 6.3 Monitor progress

7. Using the Live Camera Application

- 7.1 Open the web app
- 7.2 Register a face
- 7.3 Run recognition
- 7.4 Report a misclassification

8. Monitoring with Grafana

9. Running Tests

10. Troubleshooting

11. Glossary

1. Prerequisites & System Requirements

Before you clone the project, make sure your machine has everything listed below. Skipping any of these steps will cause errors during setup.

1.1 Hardware Requirements

Component	Minimum	Recommended
GPU	NVIDIA with 6 GB VRAM (CUDA 12.1)	NVIDIA RTX 3080 / A100
RAM	16 GB	32 GB
Disk space	50 GB free	100 GB free
CPU cores	4 cores	8+ cores
Internet	Required (download dataset & Docker images)	Broadband

1.2 Software to Install

Install each item below in the order shown. Click on any command, copy it, and paste it into your terminal.

A. Docker Engine & Docker Compose v2

Docker runs every service (Airflow, MLflow, Flask, Prometheus, Grafana) in isolated containers.

```
# Ubuntu / Debian
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODEN
AME) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Add yourself to the docker group (log out and back in after this)
sudo usermod -aG docker $USER
```

B. NVIDIA Container Toolkit

This lets Docker containers use your GPU.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-t
oolkit.list \
```

```
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' \  
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list  
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit  
sudo nvidia-ctk runtime configure --runtime=docker  
sudo systemctl restart docker
```

C. Git

```
sudo apt-get install -y git
```

1.3 Verifying Your Installation

Run these commands — each should print a version number or success message:

```
docker --version          # e.g. Docker version 25.0.3  
docker compose version    # e.g. Docker Compose version v2.24.5  
nvidia-smi                # should list your GPU  
git --version             # e.g. git version 2.43.0
```

■ ■ Troubleshooting tip

If 'docker' commands fail with 'permission denied', log out of your terminal session and log back in (the group change needs a fresh login).

2. Cloning the Repository

Cloning downloads all the project source code to your local machine. Pick a location where you have plenty of disk space (50 GB+).

```
# Navigate to wherever you want to put the project
cd ~/projects # or any folder you prefer

# Clone the repository
git clone https://github.com/DextroLaev/DA5402-MLOPS---Final-Project.git

# Enter the project directory – ALL subsequent commands run from here
cd DA5402-MLOPS---Final-Project
```

After cloning, your folder structure will look like this:

```
DA5402-MLOPS---Final-Project/
├── src/                ← Python source code (model, training, utils)
├── flask_app/          ← Web camera application
├── dags/               ← Airflow pipeline definition
├── scripts/            ← Helper shell scripts
├── monitoring/         ← Prometheus & Grafana configuration
├── data/               ← Dataset lives here (you'll create/download)
├── checkpoints/        ← Saved model weights (auto-created)
├── logs/               ← Training logs (auto-created)
├── docker-compose.yaml ← Defines all services
├── params.yaml         ← Training hyperparameters
├── requirements.txt    ← Python dependencies
├── Dockerfile.train    ← Trainer container
├── Dockerfile.api      ← Flask API container
└── Dockerfile.airflow  ← Airflow container
```

■ Success check

Type 'ls' in the terminal. You should see files like docker-compose.yaml, params.yaml, src/, flask_app/, dags/ etc.

3. Files You Must Create Yourself

The repository does not include secret/environment-specific files. You must create these yourself before starting Docker. Missing even one of them will prevent the application from starting.

■ **Critical**

Do NOT skip this section. These files contain passwords and configuration that Docker needs at startup.

3.1 The .env File (Main Configuration)

Create a file named **.env** in the root of the project folder (same directory as `docker-compose.yaml`). This file tells Docker how to connect all the services together.

```
# Run this command to open a text editor and create the file
nano .env

# Then paste the content below, edit the values, save with Ctrl+O then Ctrl+X
```

Content to paste into .env:

[illegible]

RUN_REMOTE	Set to false for local training. Set to true only if you have a separate GPU server and want to SSH into it for training.
COMPOSE_NETWORK	The Docker network name. Usually the folder name + '_mlops-net'. Leave as the default shown unless Docker warns about it.
REMOTE_SSH_USER	Your username on the remote GPU server. Only matters when RUN_REMOTE=true.

REMOTE_SSH_HOST	IP address or hostname of the remote server. Only matters when <code>RUN_REMOTE=true</code> .
REMOTE_MLFLOW_URI	URL where MLflow is reachable from the remote server. Only matters when <code>RUN_REMOTE=true</code> .
REMOTE_CODE_DIR	Folder on remote server where code should be synced. Only matters when <code>RUN_REMOTE=true</code> .
MLFLOW_MODEL_NAME	Name used to register the model in MLflow. Leave as 'SiameseFaceRecognition' unless you know what you're doing.
AIRFLOW_UID	Airflow's internal user ID. Keep at 50000.

3.2 SSH Key Setup (Remote Training Only)

If `RUN_REMOTE=true`, Airflow needs to SSH into your remote server to launch training. Skip this section entirely if you set `RUN_REMOTE=false`.

```
# Generate an SSH key pair (press Enter to accept defaults at each prompt)
ssh-keygen -t rsa -b 4096 -C 'airflow-training'
# Keys are saved to ~/.ssh/id_rsa (private) and ~/.ssh/id_rsa.pub (public)

# Copy the public key to your remote server (replace with your server details)
ssh-copy-id -i ~/.ssh/id_rsa.pub your_username@192.168.1.100

# Test that it works (should log in without asking for a password)
ssh your_username@192.168.1.100 'echo SSH works!'
```

Note

The `docker-compose.yaml` mounts `~/.ssh/id_rsa` into the Airflow containers automatically. As long as the file exists at that path, no further action is needed.

3.3 simple_auth_manager_passwords.json

Airflow 3 uses a JSON file to store hashed user passwords. Create this file in the project root:

```
# Install bcrypt temporarily to hash the password
pip install bcrypt --quiet

# Generate a hash of the password 'airflow'
python3 -c "
import bcrypt, json
password = b'airflow'
hashed = bcrypt.hashpw(password, bcrypt.gensalt()).decode()
data = {'airflow': hashed}
with open('simple_auth_manager_passwords.json', 'w') as f:
    json.dump(data, f, indent=2)
print('File created successfully')
"
```

This creates a file containing the hashed password for the default Airflow user (**username: airflow, password: airflow**).

■ Security tip

For any real-world or production deployment, change the password in the command above from 'airflow' to something strong and unique. Update AIRFLOW__CORE__SIMPLE_AUTH_MANAGER_USERS in docker-compose.yaml to match.

3.4 Monitoring Configuration Files

Create the monitoring directory and Prometheus configuration:

```
mkdir -p monitoring/grafana

# Create monitoring/prometheus.yaml
cat > monitoring/prometheus.yaml << 'EOF'
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'flask-api'
    static_configs:
      - targets: ['flask-api:2000']
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
  - job_name: 'dcgm-exporter'
    static_configs:
      - targets: ['dcgm-exporter:9400']
EOF

# Create an empty grafana.ini (Grafana uses defaults)
touch monitoring/grafana/grafana.ini
```


4. Downloading the Dataset

This application uses the **Labeled Faces in the Wild (LFW)** dataset — a collection of face photographs designed for studying unconstrained face recognition. You need to download and extract it into the correct folder.

```
# Create the data directory
mkdir -p data

# Download LFW dataset (approx. 200 MB)
cd data
wget http://vis-www.cs.umass.edu/lfw/lfw.tgz

# Extract it
tar -xzf lfw.tgz

# Rename to match what the code expects
mv lfw lfw

# Verify — you should see thousands of person folders
ls lfw/ | head -20
ls lfw/ | wc -l # should print ~5749

# Return to the project root
cd ..
```

After extraction, the structure should be:

```
data/
├── lfw/
│   ├── Aaron_Eckhart/
│   │   └── Aaron_Eckhart_0001.jpg
│   ├── Aaron_Guiel/
│   │   └── Aaron_Guiel_0001.jpg
│   ├── ... (5,749 person folders total)
│   └── Zydrunas_Ilgauskas/
│       └── Zydrunas_Ilgauskas_0001.jpg
```

■ Download alternatives

If `wget` is not available, you can also download via browser at: <http://vis-www.cs.umass.edu/lfw/lfw.tgz> — then move the file into your `data/` folder and run the `tar` command.

If you are behind a firewall, try: `curl -L http://vis-www.cs.umass.edu/lfw/lfw.tgz -o data/lfw.tgz`

5. Building & Starting All Services

Docker Compose reads the `docker-compose.yaml` file and starts every service — Postgres, MLflow, Airflow, the Flask API, Prometheus, and Grafana — all at once. The first time you run this, Docker will download base images and build the custom ones, which can take 10–20 minutes.

5.1 Build Docker Images

This step compiles the training container, the API container, and the Airflow container from the Dockerfiles in the project.

```
# Make sure you are in the project root directory
pwd    # should end with DA5402-MLOPS---Final-Project

# Build all custom images (takes 10-20 minutes first time)
docker compose build

# You should see output like:
# [+] Building 45.2s (12/12) FINISHED
# => [trainer] ...
# => [flask-api] ...
# => [airflow-webserver] ...
```

5.2 Start the Full Stack

This launches all services in the background. The `-d` flag means 'detached' — services run in the background and your terminal is free to use.

```
# Start all services
docker compose up -d

# Watch the startup logs (press Ctrl+C to stop watching — services keep running)
docker compose logs -f --tail=50
```

■ First-time startup

The `airflow-init` container runs database migrations first. The webserver only starts after that completes (can take 1-2 minutes). This is normal.

5.3 Verify All Services Are Healthy

After about 2 minutes, check that all containers are running:

```
docker compose ps

# Expected output — all services should show 'running' or 'healthy':
# NAME                                STATUS      PORTS
# postgres                            running    5432/tcp
# mlflow                              running    0.0.0.0:5000->5000/tcp
```

```
# airflow-webserver      healthy      0.0.0.0:8081->8080/tcp
# airflow-scheduler      running
# airflow-dag-processor   running
# flask-api              healthy      0.0.0.0:8080->2000/tcp
# prometheus             running      0.0.0.0:9090->9090/tcp
# grafana                running      0.0.0.0:3001->3000/tcp
```

Access each service in your browser:

Service	URL	Default Login
Airflow (pipeline control)	http://localhost:8081	airflow / airflow
MLflow (experiment tracking)	http://localhost:5000	No login needed
Flask API (camera app)	http://localhost:8080	No login needed
Prometheus (metrics)	http://localhost:9090	No login needed
Grafana (dashboards)	http://localhost:3001	admin / admin

6. Running the Training Pipeline (Airflow)

The training pipeline is managed by Apache Airflow. It handles preparing the dataset, running model training, registering the best model in MLflow, and promoting it to production. You control it through the Airflow web interface.

6.1 Open the Airflow UI

1. Open your browser and go to: **http://localhost:8081**
2. Log in with username **airflow** and password **airflow**.
3. You will see the Airflow dashboard. Find the DAG named **face_recognition_pipeline** in the list.
4. If the DAG shows as 'Paused' (toggle is grey), click the toggle switch to the left of the DAG name to enable it.

6.2 Trigger the Training DAG

The DAG runs automatically every week, but you can also start it manually:

1. Click on the DAG name **face_recognition_pipeline** to open it.
2. Click the **Trigger DAG** button (top right area of the page).
3. A dialog box appears. You can optionally change parameters like **dataset** (default: lfw) or leave everything as-is.
4. Click **Trigger** to start the pipeline.

6.3 Monitor Training Progress

The pipeline has several stages. You can watch them in the Airflow UI:

Stage	What it does	Duration
prepare_data	Validates and splits the LFW dataset into train / val / test sets	1–3 min
export_misclassified_data	Exports any user-reported misclassifications from the database (no-op on first run)	< 1 min
check_should_train	Decides whether training is needed — always proceeds on first run	< 1 min
train_model	Trains the Siamese neural network; uses Optuna for hyperparameter search	20–120 min
register_model	Registers the best model in MLflow and moves it to Staging	< 1 min
evaluate_and_promote	Compares Staging vs Production and promotes the winner	< 1 min

notify_flask	Pings the Flask API so it hot-reloads the new Production model immediately	< 1 min
api_health_check	Confirms the Flask API is still responding after the reload	< 1 min

Green boxes = completed successfully. Red boxes = failed (click the box to see the error log).

■ How to know training finished

All 8 stage boxes turn green. You can also check MLflow at <http://localhost:5000> — go to 'Models' → 'SiameseFaceRecognition'. You should see a version listed as 'Production'.

7. Using the Live Camera Application

Once a model is in Production in MLflow, the Flask web application can use it for real-time face recognition through your webcam. Access it at **http://localhost:8080**.

■ ■ Camera requirement

You need a USB or built-in webcam connected to the machine running Docker. The system uses `/dev/video0` by default. If your camera is on a different device number, you will need to edit `flask-api` in `docker-compose.yaml`.

7.1 Open the Web Application

1. Open your browser and go to: **http://localhost:8080**
2. You should see the live camera feed. If you see a black screen, your camera may not be connected or accessible.
3. The top of the page shows the current model version and recognition threshold.

7.2 Register a Face

You must register a face before the system can recognize it. Each registered person needs several photos taken for accuracy.

1. Click the **Register** button on the web interface.
2. Enter the person's name in the text field and click **Confirm**.
3. The system will automatically capture multiple face photos as the person looks at the camera.
4. Move slightly, turn your head gently, change expression — variety improves recognition accuracy.
5. When enough frames are captured, the system saves the face embeddings to its database.
6. A success message will appear. The person can now be recognized.

7.3 Run Face Recognition

1. After registering at least one person, the system is in **recognition mode** by default.
2. Look at the camera. The system will draw a bounding box around detected faces.
3. If the face matches someone in the database, their name and a confidence score appear.
4. If no match is found (or the score is below the threshold), it shows 'Unknown'.

7.4 Report a Misclassification

If the system recognizes the wrong person, you can report it. After enough reports, Airflow will automatically trigger retraining to fix the errors.

1. When the system shows a wrong prediction, click the **Report Misclassification** button.
2. Enter the **correct name** in the form.
3. Click **Submit**. The correction is saved to the database.
4. After the misclassification threshold is reached (default: 2 reports), Airflow automatically triggers a retraining run to incorporate the corrections.

■ Checking app status

Visit <http://localhost:8080/api/status> to see a JSON summary of the current mode, model version, and misclassification count.

Visit <http://localhost:8080/api/people> to see a list of all registered people.

8. Monitoring with Grafana

Grafana provides dashboards to visualize system health, GPU usage, recognition rates, and misclassification trends.

1. Open your browser and go to: **http://localhost:3001**
2. Log in with username **admin** and password **admin**. You will be prompted to change the password on first login.
3. Click **Connections** → **Data Sources** in the left sidebar, then click **Add data source**.
4. Select **Prometheus** and set the URL to: **http://prometheus:9090**
5. Click **Save & Test**. You should see 'Data source is working'.
6. Go to **Dashboards** → **New** → **Import** to create dashboards, or explore the built-in Node Exporter dashboard.

■ Available Metrics

The Flask API exposes metrics at `http://localhost:8080/metrics` in Prometheus format.

Key metrics include: `requests_total`, `uptime_seconds`, `misclassifications_total`, `face_misclassification_count`.

GPU metrics are provided by the DCGM exporter (if your GPU driver supports it).

9. Running Tests

The project includes automated tests for the model logic and the Flask API. Tests run on CPU only and do not require a GPU or a real camera.

```
# Install test dependencies (inside the project root)
pip install pytest torch torchvision flask bcrypt --quiet

# Run all tests
pytest tests/ -v

# Run only fast model unit tests
pytest tests/test_model.py -v

# Run only API integration tests
pytest tests/test_api.py -v

# Skip slow tests
pytest tests/ -v -m 'not slow'
```

■ Expected result

All tests should show 'PASSED'. A green summary line like '28 passed in 45.3s' means everything is working.

10. Troubleshooting

Below are the most common problems and how to fix them:

■ 'permission denied' when running docker commands

You are not in the docker group yet.

Fix: Run 'sudo usermod -aG docker \$USER' then log out and log back into your terminal.

Verify: Run 'groups' — you should see 'docker' in the list.

■ 'Permission denied' errors on data/ or checkpoints/ folders

Docker sometimes creates these folders as root, making them unreadable by your normal user account.

Fix: Run the command below from the project root to restore ownership:

```
sudo chown -R $USER:$USER data/ checkpoints/
```

Then restart the affected service: `docker compose restart airflow-scheduler`

■ Airflow webserver shows 'unhealthy' or fails to start

The Postgres database may not have initialized yet, or the passwords file is missing.

Fix: Run 'docker compose logs airflow-init' to see what failed.

Check that `simple_auth_manager_passwords.json` exists in the project root.

Try: 'docker compose down && docker compose up -d' to restart cleanly.

■ Training stage fails in Airflow with 'CUDA out of memory'

The GPU does not have enough VRAM for the configured batch size.

Fix: Edit `params.yaml` and reduce 'batch_size' from 32 to 16 or 8.

Save the file and re-trigger the DAG.

■ Flask app shows black camera feed

The camera device is not accessible inside the Docker container.

Fix: Run 'ls /dev/video*' to find your camera device number.

Edit docker-compose.yaml under 'flask-api → devices' and change '/dev/video0' to your actual device.

Restart: 'docker compose restart flask-api'

■ MLflow shows no experiments after training

Training may have failed silently or MLflow tracking URI is misconfigured.

Fix: Check 'docker compose logs trainer' for Python errors.

Verify MLflow is accessible at <http://localhost:5000>.

■ How to completely reset everything

To start from scratch (wipes all training data and containers):

docker compose down -v (removes containers AND volumes)

rm -rf mlruns/ mlartifacts/ checkpoints/ logs/

Then re-run: docker compose up -d

11. Glossary

Plain-English explanations of technical terms used in this manual:

Airflow	A workflow automation tool. It runs tasks (like training a model) on a schedule or when triggered, and shows you each step's status on a visual dashboard.
Docker / Docker Compose	Docker packages software into 'containers' so they run the same way on any machine. Docker Compose lets you start multiple containers together with one command.
MLflow	A tool for tracking machine learning experiments. It stores metrics (how well a model performed), saves trained model files, and lets you promote a model to 'Production'.
DAG	Directed Acyclic Graph — Airflow's term for a workflow. The <code>face_recognition_pipeline</code> DAG defines the order: <code>prepare data</code> → <code>train</code> → <code>register</code> → <code>promote</code> .
Embedding	A list of numbers that represents a face. The model converts a face photo into an embedding; similar faces have similar embeddings.
Siamese Network	A type of neural network that compares two inputs. Here it compares two face embeddings to decide if they belong to the same person.
Triplet Loss	The training method used. It teaches the model to make same-person embeddings close and different-person embeddings far apart.
Production / Staging	MLflow model stages. 'Staging' = newly trained, being evaluated. 'Production' = the live model the Flask API uses.
Prometheus	A system that collects numeric metrics (like request counts) from the Flask API every 15 seconds.
Grafana	A dashboard tool that reads from Prometheus and displays nice graphs and charts.
Misclassification	When the model says a face belongs to the wrong person. Users can report these, which triggers automatic retraining.
CUDA	NVIDIA's software that lets PyTorch use the GPU for fast math operations during training.
.env file	A plain text file containing environment variable settings. Docker Compose reads it to configure all services.

End of User Manual — Face Recognition MLOps System
For issues, open a GitHub issue at: github.com/DextroLaev/DA5402-MLOPS---Final-Project